

CURRENT AND PREVIOUS YEAR QUESTIONS (SOFTWARE)

MISTRAL SOLUTIONS

Some important links to refer:

<https://www.indiabix.com/placement-papers/mistral/>

<https://www.indiabix.com/placement-papers/mistral/4004>

<https://www.scribd.com/document/73462336/Mistral-1>

<https://www.careerride.com/view/mistral-solutions-placement-paper-selection-process-and-test-pattern-36753.aspx>

Answers for Current year Programming Questions (2025):

Round 1: Online Assessment

1. Reverse a String using Recursion

```
public class ReverseString {  
    public static String reverse(String str) {  
        if (str.isEmpty()) {  
            return str;  
        }  
        return reverse(str.substring(1)) + str.charAt(0);  
    }  
    public static void main(String[] args) {  
        String input = "hello";  
        System.out.println("Original: " + input);  
    }  
}
```

```
        System.out.println("Reversed: " + reverse(input));
    }
}
```

2. Remove Duplicates in the Linked List

```
import java.util.HashSet;

class Node {
    int data;
    Node next;
    Node(int d) {
        data = d;
        next = null;
    }
}

public class RemoveDuplicates {
    Node head;

    void insert(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        Node temp = head;
        while (temp.next != null) temp = temp.next;
```

```

        temp.next = newNode;
    }

    void removeDuplicates() {

        HashSet<Integer> seen = new HashSet<>();

        Node curr = head, prev = null;

        while (curr != null) {

            if (seen.contains(curr.data)) {

                prev.next = curr.next;

            } else {

                seen.add(curr.data);

                prev = curr;

            }

            curr = curr.next;

        }

    }

    void display() {

        Node temp = head;

        while (temp != null) {

            System.out.print(temp.data + " ");

            temp = temp.next;

        }

        System.out.println();

    }

    public static void main(String[] args) {

```

```

RemoveDuplicates list = new RemoveDuplicates();

list.insert(1);

list.insert(2);

list.insert(2);

list.insert(3);

list.insert(3);

list.insert(4);

System.out.print("Original List: ");

list.display();

list.removeDuplicates();

System.out.print("After Removing Duplicates: ");

list.display();

}

}

```

3. Decimal to Binary and find position of 1, Right Rotate

```

public class DecimalToBinary {

    public static String toBinary(int num) {

        return Integer.toBinaryString(num);

    }

    public static void findPositions(String binary) {

        System.out.print("Positions of 1 (from right, starting at 1): ");

        int len = binary.length();

        for (int i = 0; i < len; i++) {

```

```
        if (binary.charAt(len - 1 - i) == '1') {  
            System.out.print((i + 1) + " ");  
        }  
    }  
  
    System.out.println();  
}
```

// Right rotate a binary string by k

```
public static String rightRotate(String binary, int k) {  
    int n = binary.length();  
    k = k % n;  
    return binary.substring(n - k) + binary.substring(0, n - k);  
}  
  
public static void main(String[] args) {  
    int num = 23; // Example  
  
    String binary = toBinary(num);  
  
    System.out.println("Decimal: " + num);  
  
    System.out.println("Binary: " + binary);  
  
    findPositions(binary);  
  
    int rotateBy = 2;  
  
    String rotated = rightRotate(binary, rotateBy);  
  
    System.out.println("Right rotated by " + rotateBy + ": " + rotated);  
}  
}
```

Round 2 : Online Coding Round

1. Find the length of the string without using built-in functions

```
import java.util.Scanner;

public class StringLength {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");

        String str = sc.nextLine();

        int length = 0;

        try {

            while (true) {

                str.charAt(length);

                length++;

            }

        } catch (StringIndexOutOfBoundsException e) {

        }

        System.out.println("Length of the string: " + length);

        sc.close();

    }

}
```

2. Reverse the number and print the sum of all its digits

```
import java.util.Scanner;

public class ReverseAndSum {
```

```

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter a number: ");

    int num = sc.nextInt();

    int rev = 0, sum = 0, digit;

    int temp = num;

    while (temp > 0) {

        digit = temp % 10;

        rev = rev * 10 + digit;

        sum += digit;

        temp /= 10;

    }

    System.out.println("Reversed Number: " + rev);

    System.out.println("Sum of Digits: " + sum);

    sc.close();

}
}

```

3. Convert a Decimal number to binary and count the number of 1's and 0's in the binary representation.

```

import java.util.Scanner;

public class DecimalToBinaryCount {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

```

```
System.out.print("Enter a decimal number: ");

int num = sc.nextInt();

String binary = "";

int ones = 0, zeros = 0, temp = num;

if (temp == 0) {

    binary = "0";

    zeros = 1;

}

else {

    while (temp > 0) {

        int bit = temp % 2;

        binary = bit + binary;

        if (bit == 1) ones++;

        else zeros++;

        temp /= 2;

    }

}

System.out.println("Binary representation: " + binary);

System.out.println("Number of 1's: " + ones);

System.out.println("Number of 0's: " + zeros);

sc.close();

}

}
```

Round 3 : Offline Coding Round

Level 1:

1. Validate whether the given string meets password criteria:

- **At least one uppercase letter**
- **At least one lowercase letter**
- **At least one digit**
- **At least one special character**
- **No same character in adjacent**
- **Does not contain consecutive identical characters.**

Output whether each condition is satisfied or not , if not print which conditions are failing.

```
import java.util.Scanner;

public class PasswordValidator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a password: ");

        String password = sc.nextLine();

        sc.close();

        boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false,
noAdjacentSame = true;

        for (int i = 0; i < password.length(); i++) {

            char ch = password.charAt(i);

            if (Character.isUpperCase(ch)) hasUpper = true;

            if (Character.isLowerCase(ch)) hasLower = true;

            if (Character.isDigit(ch)) hasDigit = true;
```

```
        if (!Character.isLetterOrDigit(ch)) hasSpecial = true;

        if (i > 0 && password.charAt(i) == password.charAt(i - 1)) {

            noAdjacentSame = false;

        }

    }

    boolean isValid = true;

    System.out.println("\nPassword Validation Results:");

    if (hasUpper) {

        System.out.println(" Contains at least one uppercase letter");

    }

    else {

        System.out.println("Missing an uppercase letter");

        isValid = false;

    }

    if (hasLower) {

        System.out.println("Contains at least one lowercase letter");

    } else {

        System.out.println("Missing a lowercase letter");

        isValid = false;

    }

    if (hasDigit) {

        System.out.println("Contains at least one digit");

    }

    else {
```

```
        System.out.println("Missing a digit");

        isValid = false;
    }

    if (hasSpecial) {

        System.out.println(" Contains at least one special character");
    } else {

        System.out.println("Missing a special character");

        isValid = false;
    }

    if (noAdjacentSame) {

        System.out.println("No consecutive identical characters");
    } else {

        System.out.println("Has consecutive identical characters");

        isValid = false;
    }

    if (isValid) {

        System.out.println("\nPassword is VALID!");
    } else {

        System.out.println("\nPassword is INVALID!");
    }

}

}
```

2. Read a file and count the number of consecutive words containing 'e' and 'd' in it.

```
import java.io.*;

import java.util.*;

public class ConsecutiveWordsED {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter file path: ");

        String filePath = sc.nextLine();

        sc.close();

        List<String> words = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {

            String line;

            while ((line = br.readLine()) != null) {

                // Split on spaces and punctuation

                String[] tokens = line.split("[\\s\\p{Punct}]+");

                for (String word : tokens) {

                    if (!word.isEmpty()) {

                        words.add(word.toLowerCase());

                    }

                }

            }

        }

        catch (IOException e) {
```

```

        System.out.println("Error reading file: " + e.getMessage());

        return;
    }

    int count = 0;

    for (int i = 0; i < words.size() - 1; i++) {

        if (containsEandD(words.get(i)) && containsEandD(words.get(i + 1))) {

            count++;

        }

    }

    System.out.println("Number of consecutive word pairs containing 'e' and 'd': " + count);

}

private static boolean containsEandD(String word) {

    return word.contains("e") && word.contains("d");

}

}

```

Level 2:

1. Write a C program to analyze a source tree (directory) . The analysis results shall include the

below statistics about the source tree

- **Number of Directories**
- **Number of Files**
- **File extensions and their count (eg : C , C++ , Java)**
- **Number of lines in each file**
- **Number of functions in each file**

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <dirent.h>

#include <sys/stat.h>

#include <ctype.h>

#define MAX_EXT 100

typedef struct {

    char ext[20];

    int count;

} ExtensionCount;

ExtensionCount extCounts[MAX_EXT];

int extCount = 0;

int dirCount = 0, fileCount = 0;

void analyzeDirectory(const char *path);

void analyzeFile(const char *filepath);

void addExtension(const char *ext);

int countLinesInFile(const char *filepath);

int countFunctionsInFile(const char *filepath);

void addExtension(const char *ext) {

    for (int i = 0; i < extCount; i++) {

        if (strcmp(extCounts[i].ext, ext) == 0) {

            extCounts[i].count++;

        }

    }

}
```

```

        return;

    }

}

strcpy(extCounts[extCount].ext, ext);

extCounts[extCount].count = 1;

extCount++;

}

int countLinesInFile(const char *filepath) {

    FILE *fp = fopen(filepath, "r");

    if (!fp) return 0;

    int lines = 0;

    char ch;

    while ((ch = fgetc(fp)) != EOF) {

        if (ch == '\n') lines++;

    }

    fclose(fp);

    return lines;

}

int countFunctionsInFile(const char *filepath) {

    FILE *fp = fopen(filepath, "r");

    if (!fp) return 0;

    int functions = 0;

    char line[1024];

    while (fgets(line, sizeof(line), fp)) {

```

```

    char *openParen = strchr(line, '(');

    char *openBrace = strchr(line, '{');

    if (openParen && openBrace && openParen < openBrace) {

        functions++;

    }

}

fclose(fp);

return functions;

}

void analyzeFile(const char *filepath) {

    fileCount++;

    const char *dot = strrchr(filepath, '.');

    if (dot && dot != filepath) {

        addExtension(dot + 1);

    }

    int lines = countLinesInFile(filepath);

    int functions = countFunctionsInFile(filepath);

    printf("File: %s\n", filepath);

    printf("  Lines: %d\n", lines);

    printf("  Functions: %d\n\n", functions);

}

void analyzeDirectory(const char *path) {

    DIR *dir = opendir(path);

    if (!dir) return;

```

```

struct dirent *entry;

while ((entry = readdir(dir)) != NULL) {

    if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)

        continue;

    char fullPath[1024];

    snprintf(fullPath, sizeof(fullPath), "%s/%s", path, entry->d_name);

    struct stat statbuf;

    if (stat(fullPath, &statbuf) == 0) {

        if (S_ISDIR(statbuf.st_mode)) {

            dirCount++;

            analyzeDirectory(fullPath);

        } else if (S_ISREG(statbuf.st_mode)) {

            analyzeFile(fullPath);

        }

    }

}

closedir(dir);
}

int main() {

    char path[1024];

    printf("Enter directory path: ");

    scanf("%s", path);

    analyzeDirectory(path);

    printf("\n=== Analysis Report ===\n");

```

```
printf("Number of directories: %d\n", dirCount);

printf("Number of files: %d\n", fileCount);

printf("\nFile Extensions:\n");

for (int i = 0; i < extCount; i++) {

    printf("  %s: %d\n", extCounts[i].ext, extCounts[i].count);

}

return 0;

}
```